

Репликация и масштабирование. Часть 1

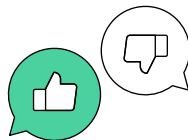


Проверка связи



Если у вас нет звука:

- убедитесь, что на вашем устройстве и на колонках включён звук
- обновите страницу вебинара (или закройте страницу и заново присоединитесь к вебинару)
- откройте вебинар в другом браузере
- перезагрузите компьютер (ноутбук) и заново попытайтесь зайти



Поставьте в чат:

-  если меня видно и слышно
-  если нет

Юрий Новиков

О спикере:

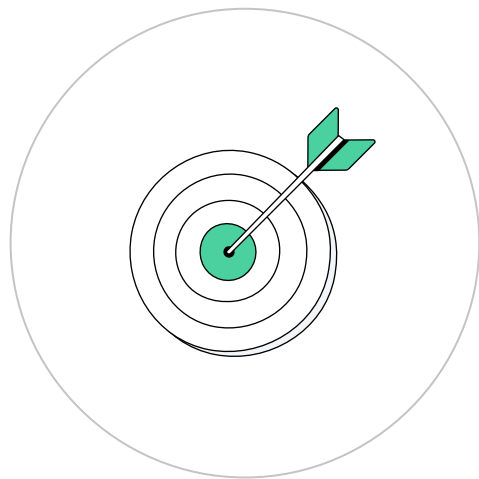
- В IT с 2000 года.
- В настоящее время ведущий разработчик в ОМК-ИТ, ранее занимал руководящие должности.
- Принимал участие в разработке веб-приложений и прикладного ПО. Одна из систем - система мониторинга городского пассажирского транспорта.
- Программирую: Delphi, C#, Php, JavaScript.
- Проектирует базы, взаимодействию с базами посредством СУБД MySql, MSQL, Oracle.
- Проводит курсы по реляционным базам данных.



Цели занятия

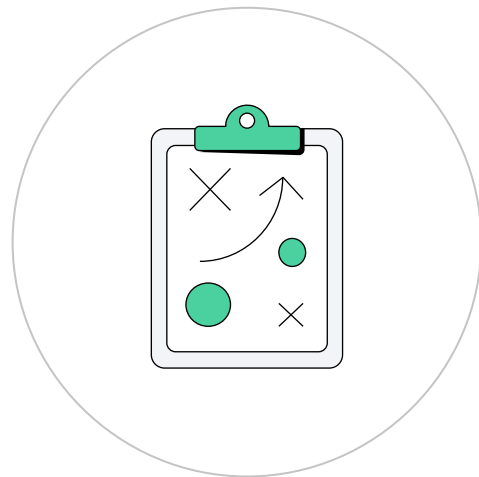
Сегодня мы:

- поговорим о репликации баз данных
- разберём, какие методы и технологии репликации существуют, как они работают
- узнаем, как настроить репликации master-slave и master-master



План занятия

- 1 Репликация
- 2 Репликация master-slave
- 3 Репликация master-master
- 4 Настройка master-slave репликации MySQL
- 5 Настройка master-slave репликации PostgreSQL
- 6 Настройка master-master репликации
- 7 Итоги
- 8 Домашнее задание



Репликация



1

Репликация

Репликация — это процесс, под которым понимается копирование данных из одного источника на другой или на множество других и наоборот.

С точки зрения базы данных, это механизм копирования базы данных и создания копий или дополнений существующих объектов

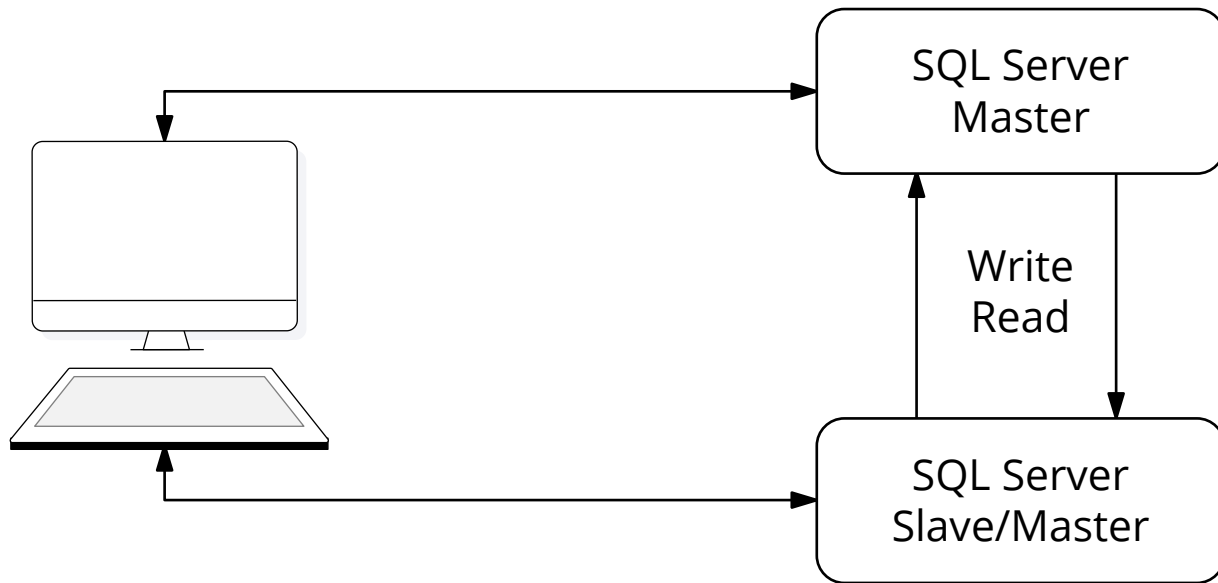
Виды репликации

- **Синхронная репликация** — изменения в основной базе и её копиях происходят в рамках одной транзакции. Запись считается завершённой только после подтверждения от всех реплик. Гарантируется, что во всех узлах в любой момент времени содержится одинаковая информация
- **Асинхронная репликация** — основная база фиксирует изменения сразу, не дожидаясь отклика от реплик. Обновления доходят до копий с небольшой задержкой, поэтому данные на разных узлах могут временно различаться

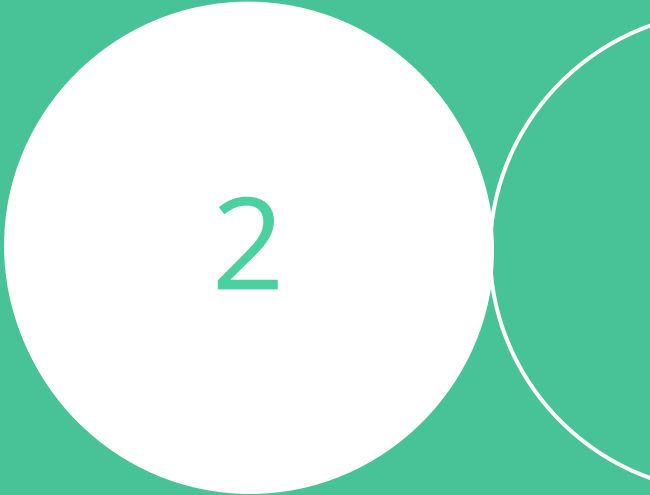
Преимущества репликации

- Повышение производительности чтения данных
- Повышение отказоустойчивости
- Распространение данных между серверами повышает надежность, доступность и скорость
- Распределение нагрузки
- Тестирование новых конфигураций
- Резервное копирование

Общая схема репликации



Репликация master-slave



2

The diagram illustrates a master-slave replication setup. It features two circular nodes. The first node, on the left, is solid blue and contains the number '1' in white. The second node, on the right, is white with a blue outline and contains the number '2' in blue. A blue arrow points from node 1 to node 2, indicating the direction of replication.

Репликация master-slave

Репликация типа master-slave часто используется для обеспечения отказоустойчивости приложений.

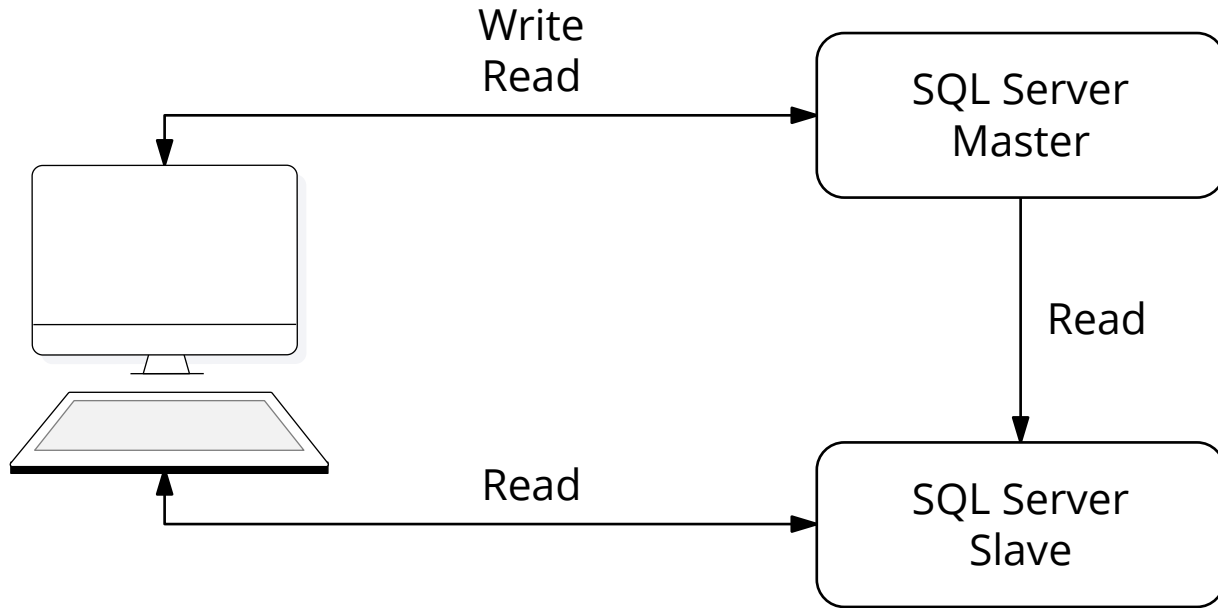
Кроме этого, она позволяет распределить нагрузку на базу данных между несколькими серверами или репликами

Репликация master-slave

Master — это основной сервер БД, куда поступают все данные. Все изменения в данных — добавление, обновление, удаление — должны происходить на этом сервере.

Slave — это вспомогательный сервер БД, который копирует все данные с мастера. С этого сервера следует читать данные. Таких серверов может быть несколько

Репликация master-slave



Репликация master-slave

Репликация выполняется по следующим шагам:

- **master** записывает изменения данных в журнал (**binary log**)
- **slave** копирует изменения двоичного журнала в свой, который называется журналом ретрансляции (**relay log**)
- **slave** воспроизводит изменения из журнала ретрансляции, применяя их к собственным данным

Репликация master-slave

Репликация бывает двух подходов: **покомандная** и **построчная**.

Покомандная репликация: в журнал master протоколируются запросы изменения данных (INSERT, UPDATE, DELETE), на slave - повторяются.

При **построчной** репликации в журнале запишутся изменения данных, то же произойдёт и на slave



**Можно ли репликацию
master-slave считать
резервным копированием
данных?**

Репликация master-master



3

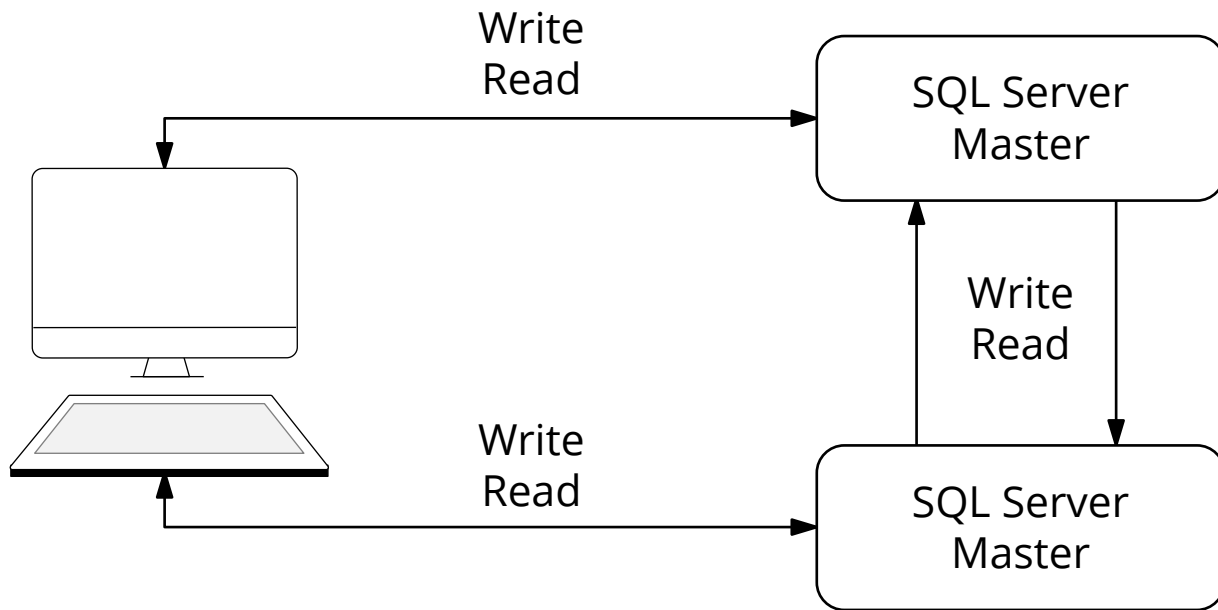
The diagram illustrates a master-master replication setup with three nodes. It consists of three overlapping circles arranged in a horizontal line. The leftmost circle is solid blue and contains the number '1' in white. The middle circle is solid blue and contains the number '2' in white. The rightmost circle is white with a blue outline and contains the number '3' in blue. This visualizes a three-node master-master replication configuration.

Репликация master-master

Репликация master-master позволяет копировать данные с одного сервера на другой. Эта конфигурация добавляет избыточность и повышает эффективность при обращении к данным.

Master-master репликации — это настройка обычной master-slave репликации, только в обе стороны. Каждый сервер является мастером и слейвом одновременно

Репликация master-master





**Возможно ли настроить
репликацию типа
slave-slave?**

Настройка master-slave репликации MySQL



4

Алгоритм

Для настройки репликации master-slave СУБД MySQL нам потребуется минимум два инстанса MySQL.

1. Создаём 2 инстанса MySQL
2. Настраиваем на каждом конфигурационный файл `my.cnf`
3. На мастере создаём пользователя для подключения реплики
4. На реплике изменяем источник данных для журнала
5. Подключаем инстансы к сети
6. Запускаем репликацию

Настройка в Docker

Наиболее наглядно можно провести настройку репликации в Docker. Представленные docker-файлы предполагают наличие в корневом каталоге файлов конфигурации.

Docker-файл для мастера: Dockerfile_master:

```
FROM mysql:9.3

# Копируем файлы конфигурации
COPY ./master.cnf /etc/mysql/conf.d/my.cnf
COPY ./master.sql /docker-entrypoint-initdb.d/start.sql

# Переменные окружения для настройки репликации
ENV MYSQL_ROOT_PASSWORD=12345

# Запускаем сервисы
CMD ["mysqld"]
```


Настройка в Docker

Docker-файл для слейва: Dockerfile_slave:

```
FROM mysql:9.3

# Копируем файлы конфигурации
COPY ./slave.cnf /etc/mysql/conf.d/my.cnf
COPY ./slave.sql /docker-entrypoint-initdb.d/start.sql

# Переменные окружения для настройки репликации
ENV MYSQL_ROOT_PASSWORD=12345

# Запускаем сервисы
CMD ["mysqld"]
```

Конфигурационные файлы

Конфигурационный файл мастера `master.cnf`:

```
[mysqld]
server-id=1
log-bin = mysql-bin
binlog_format=ROW
```

server-id — уникальный идентификатор сервера в репликационной топологии.

binlog_format — определяет формат записи изменений в бинарный лог (binary log).

Возможные значения:

- **ROW** (по умолчанию в MySQL 8.0+) — записывает изменённые строки данных
- **STATEMENT** — записывает SQL-запросы
- **MIXED** — комбинация ROW и STATEMENT

Конфигурационные файлы

Конфигурационный файл слейва `slave.cnf`:

```
[mysqld]
server-id=2
read_only = 1
```

read_only — флаг, запрещающий любые операции на запись для данной СУБД для всех пользователей, кроме пользователя, использующегося для репликации

Файлы инициализации

Файл инициализации мастера `master.sql`:

```
CREATE USER 'repl'@'%' IDENTIFIED BY 'slavepass'; -- создаём пользователя для реплики
GRANT REPLICATION SLAVE ON *.* TO 'repl'@'%'; -- выдаём права для репликации новому
пользователю
FLUSH PRIVILEGES; -- принудительно применяем изменения
```

Файл инициализации слейва `slave.sql`:

```
CHANGE REPLICATION SOURCE TO -- изменяем источник получения данных журнала в версиях
до MySQL 8.0.23 ( CHANGE MASTER TO)
SOURCE_HOST='mysql_master', -- хост мастера
SOURCE_USER='repl', -- пользователь для репликации
SOURCE_PASSWORD='slavepass', -- пароль пользователя для репликации
SOURCE_SSL=1; -- включаем ssl
START REPLICA; -- запускаем реплику
```

Запуск контейнеров

Собираем контейнеры

```
docker build -t mysql_slave -f ./Dockerfile_slave .  
docker build -t mysql_master -f ./Dockerfile_master .
```

Добавляем новую сеть

```
docker network create replication .
```

Запуск контейнеров

Запускаем контейнеры:

```
docker run --name mysql_master --net replication -p 3306:3306 mysql_master
docker run --name mysql_slave --net replication -p 3307:3306 mysql_slave
```

Контейнеры между собой соединяются сетью **replication** благодаря ключу **--net**.

Для контейнеров проброшены разные внешние порты, чтобы можно было подключиться внешним клиентом (например, DBeaver <https://dbeaver.io/download/>) и проверить работу полученного кластера

Настройка master-slave репликации PostgreSQL

A decorative graphic in the bottom right corner consisting of two overlapping circles. The left circle is white with a dark teal number '5' inside. The right circle is dark teal and partially cut off by the edge of the frame.

5

Алгоритм

Для настройки репликации master-slave СУБД PostgreSQL нам потребуется минимум два инстанса PostgreSQL.

1. Создаём 2 инстанса PostgreSQL
2. На мастере разрешаем внешние подключения в файле `pg_hba.conf`
3. На мастере создаём пользователя для подключения реплики
4. На слейве запускаем процедуру репликации `pg_basebackup`
5. Подключаем инстансы к сети

Настройка в Docker

Представленные docker-файлы предполагают наличие в корневом каталоге файлов конфигурации.

Docker-файл для мастера: Dockerfile_master:

```
FROM postgres:17
# Копируем файлы инициализации и конфигурации
COPY ./master-init.sh /docker-entrypoint-initdb.d/master-init.sh
COPY ./pg_hba_master.conf /var/lib/postgresql/data/pg_hba.conf

RUN mkdir -p /var/lib/postgresql/data
VOLUME /var/lib/postgresql/data

# Переменные окружения
ENV POSTGRES_USER=postgres
ENV POSTGRES_PASSWORD=postgres
ENV POSTGRES_DB=test_db
```

Настройка в Docker

Docker-файл для слейва: Dockerfile_slave:

```
FROM postgres:17
# Копируем файлы инициализации
COPY ./slave-init.sh /docker-entrypoint-initdb.d/master-init.sh

RUN mkdir -p /var/lib/postgresql/data
VOLUME /var/lib/postgresql/data

# Переменные окружения
ENV POSTGRES_USER=postgres
ENV POSTGRES_PASSWORD=postgres
ENV POSTGRES_DB=test_db
```

Конфигурационные файлы

Пример файла конфигурации мастера `pg_hba.conf`:

```
local    all             all                                     trust
host     all             all             127.0.0.1/32          trust
host     all             all             ::1/128               trust

host     replication     all             0.0.0.0/0            md5
local    replication     all                                     trust
host     replication     all             127.0.0.1/32          trust
host     replication     all             ::1/128               trust

host all all all scram-sha-256
```

В стандартный файл `pg_hba.conf` была внесена строка, разрешающая подключения для репликации всем пользователям из любых подсетей

```
host     replication     all             0.0.0.0/0            md5
```

Файлы инициализации

Файл инициализации мастера `master.sql`:

```
psql -v ON_ERROR_STOP=1 --username "$POSTGRES_USER" --dbname "$POSTGRES_DB" <<-EOSQL
CREATE USER replicator WITH REPLICATION ENCRYPTED PASSWORD 'replicatorpass';
ALTER SYSTEM SET wal_level = 'replica';
ALTER SYSTEM SET max_wal_senders = 10;
ALTER SYSTEM SET wal_keep_size = 1024;
ALTER SYSTEM SET hot_standby = on;
EOSQL
# Применяем изменения
pg_ctl restart
```

Создаёт пользователя **replicator** с правами на репликацию (REPLICATION).

Устанавливает пароль **replicatorpass** (в зашифрованном виде, благодаря ENCRYPTED).

wal_level = 'replica' — включает запись достаточного количества информации в WAL, чтобы standby-сервер мог читать и применять изменения.

max_wal_senders — определяет, сколько одновременно может быть подключений для репликации.

wal_keep_size = 1024 — сохраняет WAL-файлы размером минимум 1024 МБ, чтобы standby-сервер мог успеть их забрать.

hot_standby = on — позволяет standby-серверу принимать запросы на чтение (пока он реплицирует данные с мастера)

Файлы инициализации

Файл инициализации слейва `slave.sql`:

```
PGPASSWORD=replicatorpass pg_basebackup -h postgres-master -U replicator -p 5432 -D  
/var/lib/postgresql/data -Fp -Xs -P -R -S slave1 -C
```

- h **postgres-master** — указывает адрес сервера-источника (primary).
- U **replicator Пользователь** — подключается от имени пользователя replicator (созданного ранее).
- p **5432** — стандартный порт, на котором работает мастер.
- D **/var/lib/postgresql/data** — куда сохранить копию данных (обычно это каталог data standby-сервера).
- Fp — копирует файлы «как есть» (вместо архива .tar).
- Xs — параллельно с бэкапом потоково передаёт WAL-логи (stream).
- P — показывает прогресс копирования в реальном времени.
- R — создаёт standby.signal и настраивает postgresql.auto.conf для реплики.
- S **slave1** — создаёт слот репликации с именем slave1 (для отслеживания прогресса).
- C — создаёт слот репликации перед началом копирования (если слота не существует)

Запуск контейнеров

Собираем контейнеры

```
docker build -t pg_master -f ./Dockerfile_master .  
docker build -t pg_slave -f ./Dockerfile_slave .
```

Добавляем новую сеть

```
docker network create replication .
```

Запуск контейнеров

Запускаем контейнеры:

```
docker run --name postgres-master --net replication -p 5432:5432 -v ./data:/var/lib/postgresql/data -d pg_master  
docker run --name postgres-slave --net replication2 -p 5433:5432 -v ./data:/var/lib/postgresql/data -d pg_slave
```

Контейнеры между собой соединяются сетью **replication** благодаря ключу **--net**.

Ключ **-v** монтирует внешний том с данным postgres

Настройка master-master репликации



6

Алгоритм

Для настройки репликации master-master СУБД MySQL нам потребуется минимум два инстанса MySQL

1. Создаём 2 инстанса MySQL master-1 и master-2
2. Настраиваем на каждом конфигурационный файл `my.cnf`
3. На master-1 создаём пользователя для подключения реплики и изменяем источник данных для журнала на master-2
4. На master-2 создаём пользователя для подключения реплики и изменяем источник данных для журнала на master-1
5. Подключаем инстансы к сети
6. Запускаем репликацию

Настройка в Docker

Наиболее наглядно можно провести настройку репликации в Docker.
Docker-файл для первого мастера: Dockerfile_master-1:

```
FROM mysql:9.3

# Копируем файлы конфигурации
COPY ./master-1.cnf /etc/mysql/conf.d/my.cnf
COPY ./master-1.sql /docker-entrypoint-initdb.d/start.sql

# Переменные окружения для настройки репликации
ENV MYSQL_ROOT_PASSWORD=12345

# Запускаем сервисы
CMD ["mysqld"]
```

Настройка в Docker

Docker-файл для второго мастера: Dockerfile_master-2:

```
FROM mysql:9.3

# Копируем файлы конфигурации
COPY ./master-2.cnf /etc/mysql/conf.d/
COPY ./master-2.sql /docker-entrypoint-initdb.d/start.sql

# Переменные окружения для настройки репликации
ENV MYSQL_ROOT_PASSWORD=12345

# Запускаем сервисы
CMD ["mysqld"]
```

Конфигурационные файлы

Конфигурационный файл мастера `master-1.cnf`:

```
[mysqld]
server_id = 1
log-bin = mysql-bin
binlog_format=ROW
```

server-id — уникальный идентификатор сервера в репликационной топологии.

binlog_format — определяет формат записи изменений в бинарный лог (binary log).

Возможные значения:

- **ROW** (по умолчанию в MySQL 8.0+) — записывает изменённые строки данных
- **STATEMENT** — записывает SQL-запросы
- **MIXED** — комбинация ROW и STATEMENT

Конфигурационные файлы

Конфигурационный файл мастера `master-2.cnf`:

```
[mysqld]
server_id = 2
binlog_format=ROW
log-bin
```

server-id — уникальный идентификатор сервера в репликационной топологии.

binlog_format — определяет формат записи изменений в бинарный лог (binary log).

Возможные значения:

- **ROW** (по умолчанию в MySQL 8.0+) — записывает изменённые строки данных
- **STATEMENT** — записывает SQL-запросы
- **MIXED** — комбинация ROW и STATEMENT

Файлы инициализации

Файл инициализации мастера `master-1.sql`:

```
CREATE USER 'repl_1'@'%' IDENTIFIED BY 'pass';
GRANT REPLICATION SLAVE ON *.* TO 'repl_1'@'%;

CHANGE REPLICATION SOURCE TO
SOURCE_HOST='master-2',
SOURCE_USER='repl_2',
SOURCE_PASSWORD='pass',
SOURCE_SSL=1;

START REPLICA;
```

Файлы инициализации

Файл инициализации мастера `master-2.sql`:

```
CREATE USER 'repl_2'@'%' IDENTIFIED BY 'pass';  
GRANT REPLICATION SLAVE ON *.* TO 'repl_2'@'%;  
  
CHANGE REPLICATION SOURCE TO  
SOURCE_HOST='master-1',  
SOURCE_USER='repl_1',  
SOURCE_PASSWORD='pass',  
SOURCE_SSL=1;  
  
START REPLICA;
```

Запуск контейнеров

Собираем контейнеры

```
docker build -t master-1 -f ./Dockerfile_master-1 .  
docker build -t master-2 -f ./Dockerfile_master-2 .
```

Добавляем новую сеть

```
docker network create replication_mm
```


Запуск контейнеров

Запускаем контейнеры:

```
docker run --name master-1 --net replication_mm -p 3306:3306 -d master-1
docker run --name master-2 --net replication_mm -p 3307:3306 -d master-2
```

Контейнеры между собой соединяются сетью `replication_mm` благодаря ключу `--net`.

Для контейнеров проброшены разные внешние порты, чтобы можно было подключиться внешним клиентом (например, DBeaver <https://dbeaver.io/download/>) и проверить работу полученного кластера

Тестирование

SQL запросы на master:

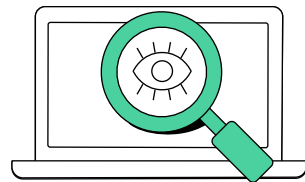
```
CREATE database world;  
SHOW databases;  
USE world;  
INSERT INTO city (Name, CountryCode, District, Population) VALUES  
'Test-Replication', 'ALB', 'Test', 42);
```

SQL запросы на slave:

```
SHOW databases; — должна отобразиться база данных world созданная на мастере  
USE world;  
SHOW tables; — должна отобразиться таблица city созданная на мастере  
SELECT * FROM city ORDER BY ID DESC LIMIT 1; — получение данных добавленных  
на мастере
```

Демонстрация работы

Произведём настройку базы данных для работы в режиме репликации



Итоги

7

Итоги

Сегодня мы:

- рассмотрели принципы репликации
- узнали, как настраивать репликацию в различных режимах
- узнали, как настраивать репликацию для СУБД MySQL и PostgreSQL



Домашнее задание

8

Домашнее задание

Давайте посмотрим ваше домашнее задание.

- Вопросы по домашней работе задавайте в разделе "Вопросы по заданию"
- Задачи можно сдавать по частям
- Зачёт по домашней работе проставляется после того, как приняты все задачи



Репликация и масштабирование. Часть 1

